How Far Can Labels Go? Benchmark Difficulty Is the Bottleneck of Cold-Start Predictive Evaluation

Hung-Kai Huang
National Yang Ming Chiao Tung University
kkk61.ai15@nycu.edu.tw

Abstract

Predictive evaluation estimates whether an AI system will solve a benchmark item from a few observed outcomes. We study the benchmark-level cold start of the Predictive AI Evaluation Competition, where every test item comes from a benchmark absent from the training data and at most 31 outcomes are revealed per system–benchmark pair. Because each formative submission is scored on a new random sample, raw scores mix method quality with sample difficulty. We recover each pair's success rate from its budget-0 Brier score and calibration error and build a proxy that compares label-only predictors on 69 real test pairs and reproduces our submissions' scores within label noise. On this proxy, hierarchical Beta-Binomial shrinkage (HBS), which requires no training, improves on the official baseline by 0.028 Brier ALC and outperforms a Bayes-optimal predictor that uses only each pair's own labels. More than half of its remaining gap to an oracle comes from not knowing a new benchmark's difficulty. Item-level LLM difficulty estimates did not transfer to the test benchmarks, but averaging Qwen3.8-27B's ratings over a benchmark's items gives a prior that recovers about two fifths of this component, a further 0.007 on the seven test benchmarks. Seven benchmarks are too few to establish this gain for new ones. On a fresh test sample, it lowered the budget-0 Brier score from 0.224 to 0.190. HBS with this prior is our final submission.

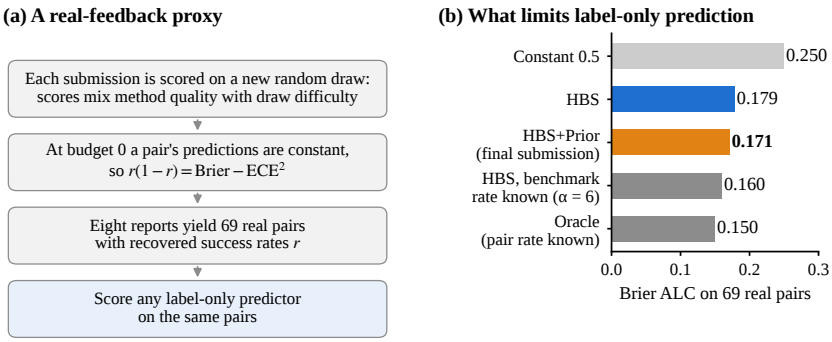


Figure 1: Overview. (a) Recovering each pair's success rate from formative feedback allows all label-only predictors to be scored on the same 69 real pairs. (b) Brier ALC on these pairs. Knowing each benchmark's success rate in advance ($\alpha = 6$) would remove most of the gap between HBS and the oracle; the LLM benchmark prior of our final submission recovers two fifths of that improvement.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Predictive AI Evaluation Competition.

1 Introduction

Evaluating every AI system on every benchmark item is increasingly expensive, particularly for agentic benchmarks, in which a single attempt may involve a long tool-use trajectory. Predictive evaluation reduces this cost by estimating the probability that a system solves an item from attributes of the system and the item and from a few observed outcomes [Truong et al., 2025, Maia Polo et al., 2024]. Most existing methods calibrate item parameters on a population of systems that have already been evaluated on the same benchmark.

The Predictive AI Evaluation Competition (PAIEC) [Stanford AI Measurement Science, 2026] removes this assumption. Every test item comes from a benchmark absent from the public training pool, a predictor observes at most 31 outcomes per subject–benchmark pair, and predictions are scored by the Brier score averaged over label budgets, an area under the learning curve (Brier ALC). We call this setting *benchmark-level cold start*: before the first label arrives, nothing measured on the test benchmark is available, and afterwards too little is available to calibrate item parameters.

Developing methods for this setting also raises a measurement problem. Formative feedback for each submission is computed on a newly sampled set of pairs, and these samples differ widely in difficulty. Across the eight samples from which we build our proxy, an oracle that knows each pair's success rate scores between 0.099 and 0.183, whereas the methods we wished to compare differ by less than 0.01. Raw scores can therefore rank methods incorrectly. A variant of our method with a subject-ability adjustment scored 0.164 and HBS scored 0.188, yet on identical pairs the variant is worse by 0.026 (Section 6.1).

We make three contributions (Figure 1):

1. **The bottleneck is the benchmark prior.** More than half of the gap between a strong label-only predictor and an oracle comes from not knowing a new benchmark's difficulty before its labels arrive (Section 6.2). Item-level difficulty estimates from open LLMs did not transfer to the test benchmarks, but averaged over a benchmark's items, the ratings of a recent open model recover about two fifths of the missing prior (Sections 6.3 and 6.4).
2. **A real-feedback proxy.** When the budget-0 prediction is constant within a pair, the pair's success rate follows from its budget-0 Brier score and expected calibration error (ECE) (Proposition 1). Our eight reports yield 69 pairs on which label-only predictors can be compared directly, and the proxy reproduces our submissions' scores within label noise (Section 5).
3. **A strong label-only predictor.** Hierarchical Beta-Binomial shrinkage (HBS) pools labels across subjects and benchmarks and needs no training or pretrained model (Section 4). On the proxy it outperforms a Bayes-optimal predictor restricted to each pair's own labels, and no label-only variant we tested improves on it by more than 0.0004 (Section 6.1).

Our final submission combines HBS with the LLM benchmark prior (HBS+Prior).[1] Code to reproduce all results is available at `https://github.com/kay0601/how-far-can-labels-go`.

2 Related work

Efficient and predictive evaluation. Item response theory (IRT) [Lord, 1980] underlies most work on cheaper evaluation: tinyBenchmarks [Maia Polo et al., 2024] estimate accuracy from about 100 curated items, ATLAS [Li et al., 2026] selects items by Fisher information, and Truong et al. [2025] predict item difficulty from content. All need item parameters from systems already evaluated on the same benchmark, which benchmark-level cold start lacks.

Extending evaluation to new benchmarks. Habba et al. [2026] place new benchmarks on an existing IRT scale using anchor items answered by many models, which one or two subjects per test benchmark cannot provide. Observational scaling laws [Ruan et al., 2024] and BenchPress [Zeng and Papailiopoulos, 2026] predict a model's scores on held-out benchmarks from its other scores. Here, the benchmarks rather than the models are new.

[1] Codabench submission 978186. HBS alone, which needs neither a GPU nor a pretrained model, is submission 952533.

Predicting difficulty from content. Krsteski and Meyer [2026] predict task difficulty across 17 agentic benchmarks from reasoning-trace entropy, embeddings and metadata. Their leave-one-benchmark-out Spearman correlation of 0.225 is close to our 0.24 to 0.28 (Section 6.3), and they call transfer to unseen benchmarks open. Zhou et al. [2025] annotate 18 demand rubrics per item with GPT-4o and train an assessor for each AI system, reaching an AUROC of 0.747 on held-out benchmarks, but need a proprietary annotator and per-system training data, whereas the test subjects are new. Park et al. [2025] predict benchmark scores from descriptions of the task and setup.

Shrinkage and probabilistic forecasts. HBS is an empirical-Bayes estimator [Efron and Morris, 1975]. Predictions are scored with the Brier score [Brier, 1950] and expected calibration error [Guo et al., 2017], and our proxy uses the single-bin case of Murphy's decomposition [Murphy, 1973].

Concurrent work. Another PAIEC entry [aghasalim, 2026] uses a shrunk logistic predictor and compares versions by regressing each pair's Brier ALC on its budget-0 Brier score over 42 submissions, whereas our proxy compares methods on identical pairs. It also finds that item-text features and sentence embeddings do not help on new benchmarks.

3 Problem setup

Task. A *subject* s is an AI system together with its evaluation settings (model, provider, dates, harness, reasoning effort). An *item* i belongs to a benchmark $b(i)$ and is described by its text, structured features (`item_features`), interactors and an anonymous benchmark identifier. The outcome $y_{si} \in \{0, 1\}$ records whether s solved i. A *pair* $j = (s, b)$ is a subject–benchmark combination with at least 80 items, split evenly into acquisition items A_j and evaluation items E_j; its *success rate* is $r_j = |E_j|^{-1} \sum_{i \in E_j} y_{si}$. A *draw* D is the set of pairs sampled to score one submission. At label budget $t \in \mathcal{T} = \{0, 1, 3, 7, 15, 31\}$, the outcomes of up to t items from A_j have been revealed for every pair $j \in D$; we write $\mathcal{L}_t$ for all labels revealed in the draw. A predictor $f(s, i; \mathcal{L}_t) \in [0, 1]$ is called for every evaluation item at every budget, and every call receives all of $\mathcal{L}_t$. By default, labels are acquired in random order, but a submission may supply its own acquisition policy.

Metric. At budget t, pair j is scored by the Brier score of its evaluation items, and Brier ALC averages a weighted sum over budgets across the pairs of a draw:

$$\mathrm{Brier}_j(t) = \frac{1}{|E_j|} \sum_{i \in E_j} \big(f(s, i; \mathcal{L}_t) - y_{si}\big)^2, \qquad \mathrm{ALC}(f; D) = \frac{1}{|D|} \sum_{j \in D} \sum_{t \in \mathcal{T}} w_t \, \mathrm{Brier}_j(t), \tag{1}$$

with $w_0 = w_{31} = 0.1$ and $w_1 = w_3 = w_7 = w_{15} = 0.2$, so that the weights sum to one. Lower is better. For each pair and budget, formative feedback reports $\mathrm{Brier}_j(t)$ and the expected calibration error over ten equal-width bins,

$$\mathrm{ECE}_j(t) = \sum_{m=1}^{10} \frac{|E_{jm}|}{|E_j|} \left| \frac{1}{|E_{jm}|} \sum_{i \in E_{jm}} \big(f(s, i; \mathcal{L}_t) - y_{si}\big) \right|, \tag{2}$$

where $E_{jm} \subseteq E_j$ contains the items whose prediction falls in bin m.

Data. Benchmarks were assigned at random to disjoint training and test pools before curation. We use the binary benchmarks of the public training pool of measurement-db [AIMS Foundations, 2026], four of which have several subjects per item (Table 4 in Appendix C). Each formative submission is scored on a draw of at most 1,000 subject–item pairs. The eight draws from which we build the proxy contain 8 or 9 pairs each, mostly with one subject per benchmark, from seven test benchmarks in total. The final ranking uses a common hidden test subset.

Offline simulator. Methods that use item text, and those developed before the proxy, are evaluated on 400 draws simulated from the four multi-subject training benchmarks like test draws: about eight pairs per draw, mostly one subject per benchmark, 88 to 230 items per pair and random acquisition. Methods that use item text are selected leave-one-benchmark-out (LOBO).

4 Hierarchical Beta-Binomial shrinkage

HBS estimates a success rate for each pair by shrinking every level of a three-level hierarchy (global, benchmark, pair) toward the level above it, and then applies two small item-level corrections. HBS computes every quantity from $\mathcal{L}_t$ within the call and keeps no state between calls. Apart from its hyperparameters, nothing is fitted to training data.

Pair estimate. For a target pair (s, b), let k_{sb} and n_{sb} be the subject's successes and labels on b in $\mathcal{L}_t$, let $K_{-s,b}$ and $N_{-s,b}$ be those of all other subjects on b, and let K_{-b} and N_{-b} be those on all other benchmarks. HBS computes

$$g_b = \frac{K_{-b} + \kappa_0\,\mu_0}{N_{-b} + \kappa_0}, \qquad \pi_{sb} = \frac{K_{-s,b} + \kappa_1\,g_b}{N_{-s,b} + \kappa_1}, \qquad \bar{p}_{sb} = \frac{k_{sb} + \alpha\,\pi_{sb}}{n_{sb} + \alpha}. \tag{3}$$

Each step is the posterior mean of a Beta-Binomial model whose prior has its mean at the level above and a strength of κ_0, κ_1 or α pseudo-labels. The global rate g_b is shrunk strongly toward μ_0 because success rates differ widely across benchmarks, so labels on other benchmarks say little about b. The benchmark rate π_{sb} excludes the target subject's own labels, which already enter $\bar{p}_{sb}$ and would otherwise be counted twice. At budget 0, $\mathcal{L}_0$ is empty and HBS predicts μ_0.

Item and group corrections. Let $R(i)$ be the labels that other subjects received on the same item i, and $G(i)$ the labels, from any subject including s, on items of the same benchmark that share i's feature group, given by the non-numeric fields of `item_features` (Appendix C), which identify, for example, a contest, website, paper or programming language. For a multiset S of labels (s', i', y), let $\Delta(S) = \sum_{(s',i',y)\in S}(y - \bar{p}_{s'b})$ be the summed residual of each label against the estimate of Equation (3) for its own pair (s', b). The prediction is

$$\hat{p}_{si} = \mathrm{clip}_{[\epsilon,\,1-\epsilon]}\Big(\bar{p}_{sb} + \gamma_I\,\frac{\Delta(R(i))}{|R(i)| + \kappa_I} + \gamma_G\,\frac{\Delta(G(i))}{|G(i)| + \kappa_G}\Big). \tag{4}$$

The item correction needs another subject's label on item i; removing it raises the simulated Brier ALC by only 0.0002. The group correction transfers within-benchmark difficulty structure, such as a hard contest, once its labels arrive. Because $G(i)$ includes the target pair's own labels, and labels on item i also enter $R(i)$, the group correction also weakens the shrinkage of $\bar{p}_{sb}$ when a benchmark's items share one group (Appendix C), and the hyperparameters were tuned with this behavior.

Hyperparameters. We use $\mu_0 = 0.35$, $\kappa_0 = 100$, $\kappa_1 = \alpha = 3$, $\kappa_I = 1$, $\gamma_I = 1$, $\kappa_G = 5$, $\gamma_G = 0.75$ and $\epsilon = 0.01$ (Table 5 in Appendix C). They were chosen with the offline simulator (Section 3) and the reports of the smoothed mean and HBS+Ability (Appendix A), and re-tuning them on the proxy does not help (Section 6.1). HBS is about 170 lines of plain Python.

LLM benchmark prior (HBS+Prior). Our final submission replaces the constant μ_0 with a benchmark-specific prior estimated from item text. Each item to be predicted is rated by Qwen3.8-27B [Qwen Team, 2026b], quantized to 4 bits [Dettmers et al., 2023], with an anchored prompt that shows twelve items from the four multi-subject training benchmarks together with their observed success rates (Appendix B). The rating is $\rho_i = (\mathbb{E}[d] + 0.5)/10$, where $\mathbb{E}[d]$ is the expected digit under the model's next-token distribution over the digits 0 to 9. With I_b the items rated so far on benchmark b, the prior

$$\mu_b = \frac{\sum_{i\in I_b}\rho_i + K_0\,\mu_0}{|I_b| + K_0}, \qquad K_0 = 5, \tag{5}$$

replaces μ_0 in the global rate g_b of Equation (3). Labels on other benchmarks still dilute μ_b through g_b, mainly at large budgets, where the pair's own labels dominate; fixing g_b at μ_b would gain only 0.0006 (Appendix E). Unlike HBS, HBS+Prior keeps state. It caches ratings across calls, budgets and subjects, so μ_b averages all rated items of the benchmark from budget 1 onward and grows item by item at budget 0, where its predictions therefore vary within a pair (Appendix C).

5 Evaluating methods with formative feedback

To compare methods on the same pairs, we recover the success rate of every pair in our feedback reports and build a proxy on these rates.

Table 1: Real Brier ALC and the proxy's expectation for the same predictor and pairs; SD is the spread due to label noise alone, and differences are computed before rounding. HBS+Ability's rates are inferred (Section 5).

Submission	Real	Proxy	SD	Real − proxy	z
Smoothed mean	0.1829	0.1899	0.0069	−0.0070	−1.0
HBS+Ability	0.1639	0.1741	0.0076	−0.0103	−1.3
HBS	0.1883	0.1946	0.0077	−0.0063	−0.8

5.1 Recovering pair success rates

Proposition 1. *If the budget-0 prediction is a constant p_j on the evaluation items of pair j, then* $\mathrm{Brier}_j(0) = r_j(1-r_j) + (p_j - r_j)^2$ *and* $\mathrm{ECE}_j(0) = |p_j - r_j|$. *Consequently,*

$$r_j(1-r_j) = \mathrm{Brier}_j(0) - \mathrm{ECE}_j(0)^2. \tag{6}$$

Proof. Because $y_{si}^2 = y_{si}$, $\mathrm{Brier}_j(0) = |E_j|^{-1} \sum_{i \in E_j} (p_j^2 - 2p_j y_{si} + y_{si}) = p_j^2 - 2p_j r_j + r_j = r_j(1-r_j) + (p_j - r_j)^2$. All predictions fall into the same bin, so Equation (2) reduces to $|p_j - r_j|$. □

The condition holds for HBS, and for any predictor whose budget-0 output depends only on the subject and the benchmark, because no labels are available at budget 0. Equation (6) determines r_j up to the reflection $r_j \leftrightarrow 1 - r_j$. When $p_j \neq 1/2$ is known, $\mathrm{Brier}_j(0) = p_j^2 + (1 - 2p_j)\, r_j$ gives r_j directly. The prediction is unknown for 43 pairs: 34 from four early submissions whose code we did not retain, and 9 from HBS+Ability, whose prediction is constant within a pair but depends on subject metadata that the logs anonymize. For these, we choose the root closer to the rates of the same benchmark in the reports whose prediction is known. On the 26 pairs with known roots, from the reports of the smoothed mean, HBS and HBS+ItemLLM, the rule is correct for 25 (Appendix E).

Oracle limit of a draw. A predictor that knows every r_j and predicts it at every budget scores $\mathrm{Brier}_j(t) = r_j(1-r_j)$, so by Equation (1) its Brier ALC is

$$\bar{v}(D) = \frac{1}{|D|} \sum_{j \in D} r_j(1-r_j). \tag{7}$$

By Proposition 1, no predictor that is constant within each pair can score below $\bar{v}(D)$. We call $\bar{v}(D)$ the *oracle limit* of the draw. Across our eight reports it ranges from 0.099 to 0.183 (Figure 3 in Appendix A), and the two best raw scores were obtained on the two easiest draws.

5.2 The proxy

The proxy scores a label-only predictor f on the 69 recovered pairs. For each report, we draw label sequences $y_{j1}, \ldots, y_{j31} \sim \mathrm{Bernoulli}(r_j)$ independently for its pairs, reveal the first t labels of every pair at budget t, and query f with item inputs that carry only the subject and the benchmark identifier. Since the outcomes on E_j average to r_j, the Brier score of a prediction $p_j(t)$ is exactly $r_j(1-r_j) + (p_j(t) - r_j)^2$, and averaging over label sequences gives f's expected Brier ALC on the report. The pairs of a report are simulated jointly, so pooling behaves as on the real draw. Parameters are selected leave-one-report-out, and 95% intervals come from resampling whole reports.

Validity. For the three submissions whose code we retained (the smoothed mean, HBS+Ability and HBS; Appendix A), Table 1 compares the real score with the proxy's expectation for the same predictor on the same pairs, and with the spread due to label noise. Every real score lies 0.006 to 0.010 below its expectation, within 1.3 standard deviations. The item and group corrections, which the proxy disables, account for about 0.0015 of this difference, and the rest is consistent with favorable label sequences (Appendix E). At the pair level, 24 of the 25 pairs with label noise lie within two standard deviations. We therefore use it as our primary criterion for label-only predictors.

Table 2: Label-only alternatives to HBS. Positive changes in Brier ALC are worse; intervals are 95% intervals from resampling whole reports with the inferred roots held fixed, and for the single-draw HBS+Ability row, SD is that of the paired difference. HBS scores 0.179 on the proxy.

Alternative to HBS	Selection	Change in Brier ALC
Real-feedback proxy, 69 pairs		
Official empirical-mean baseline	reference	$+0.0282$ $[+0.0234, +0.0324]$
Smoothed mean	fixed	$+0.0028$ $[+0.0013, +0.0043]$
Benchmark pooling removed ($\kappa_1 \to \infty$)	fixed	$+0.0030$ $[+0.0015, +0.0045]$
Bayes-optimal own-label predictor	reference	$+0.0024$ $[+0.0014, +0.0034]$
Mixture prior with a near-zero component	leave-one-report-out	$+0.0004$ $[-0.0004, +0.0008]$
Re-tuned μ_0, α, κ_1	leave-one-report-out	$+0.0006$ $[-0.0008, +0.0015]$
HBS+Ability, on its own draw	fixed	$+0.0265$ (SD 0.0045)
Offline simulator, 400 draws		
Stratified label acquisition	fixed	-0.0004 (SE 0.0001)
Group correction removed	fixed	$+0.0011$ (SE 0.0001)

5.3 Measuring an LLM prior on the test benchmarks

The proof of Proposition 1 applies at any budget at which predictions are constant within each pair, in which case $\mathrm{ECE}_j(t) = |p_j(t) - r_j|$. We used this in one measurement submission, not intended as a final entry, to estimate the LLM prior of Equation (5) on the test benchmarks. At budget 0 it rated and cached every item but predicted the constant 0.35, which identifies r_j and fixes μ_b from budget 1. At budget 1 it predicted the prior μ_b, which yields $|\mu_b - r_j|$ but not its sign, and at budget 3 it predicted $\mu_b + 0.1$, which breaks this symmetry. In simulation, this procedure recovers μ_b exactly, and in the real run every pair's budget-1 scores fit a constant prediction. Because benchmark identifiers are fixed across submissions, the proxy can then score HBS+Prior on all 69 pairs.

Use of formative feedback. The proxy and the measurement submission rely only on our own formative feedback, which the competition provides to guide development. We used the recovered rates and priors only to compare methods offline. No recovered value is written into any submitted program, we made no attempt to identify the subjects or benchmarks behind the anonymous identifiers, and we do not release these values.

6 Experiments

6.1 Label-only prediction

On the 69 proxy pairs, HBS scores 0.179, compared with 0.207 for the organizers' empirical-mean baseline, 0.250 for a constant prediction of 0.5 and 0.150 for the oracle (Table 2). Pooling the labels of other subjects on the same benchmark matters. Removing it costs 0.0030, as much as replacing HBS with the simpler smoothed mean (0.0028), although pooling changes predictions only for the 26 pairs that share their benchmark with another subject in the same draw. A Bayes-optimal predictor that knows the distribution of the other pairs' success rates but sees only the target pair's labels is worse than HBS by 0.0024, although estimating that distribution on the same reports favors it. Neither a mixture prior with a near-zero component nor re-tuning μ_0, α and κ_1 on held-out reports improves on HBS, and stratified label acquisition gains only 0.0004 in simulation. These conclusions also hold on the six reports that did not inform the design of HBS and on the 26 pairs whose root is known, where the differences to the baseline, the smoothed mean, HBS without pooling and the own-label predictor are $+0.024$, $+0.005$, $+0.005$ and $+0.003$ (Appendix E).

Subject-ability adjustment. HBS+Ability shifts the benchmark prior in logit space by a subject ability fitted on the training data (Appendix A). It helped in simulation, but on real feedback it raised the budget-0 Brier score to 0.326, worse than predicting 0.5, and lost 0.026 to HBS on its own draw. The training pool contains many older systems, whereas the test subjects are recent systems on harder benchmarks.

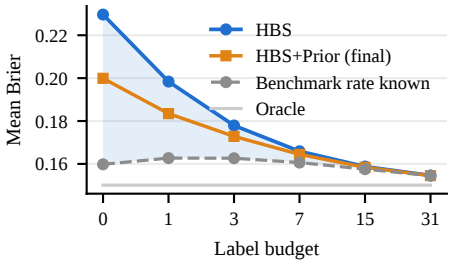


Figure 2: Mean Brier score per label budget on the 69 real pairs. Knowing each benchmark's rate ($\alpha = 6$; with $\alpha = 3$ the score is 0.162 instead of 0.160) removes most of the error of HBS at low budgets, and the LLM prior recovers part of it.

6.2 Where the remaining error lies

On the 69 pairs, HBS is 0.029 above the oracle. To locate this gap, we gave HBS each benchmark's mean success rate in advance, estimated from the other reports, in place of μ_0. With the pair-prior strength of HBS ($\alpha = 3$), this lowers its score to 0.162 and closes 58% of the gap (95% interval 54 to 61%). With $\alpha = 6$, which suits an accurate prior, it closes 65% (60 to 69%). On the 26 pairs with known roots, $\alpha = 3$ and $\alpha = 6$ close 54% and 56%. The loss is concentrated at budgets 0 to 3, where HBS relies on its global prior, and vanishes by budget 31 (Figure 2). The remainder reflects differences between subjects on the same benchmark and the error in each benchmark's estimated rate. This diagnostic uses rates recovered from test feedback and could not be part of a submission.

6.3 Can item text close the gap?

Before any labels arrive, item text is the only information available about a new benchmark. We evaluated open LLMs that fit on the competition's hardware, holding out one of the four multi-subject training benchmarks at a time and using each item's success rate averaged over subjects as the target (prompts in Appendix B; further details in Appendices C and F).

Item difficulty within a benchmark. We prompted Qwen3-8B and Qwen3-32B [Yang et al., 2025] to rate 5,322 training items and fitted a ridge-regression probe on the last prompt token's hidden state. On the held-out benchmark, the probe ranks items with an average Spearman correlation of 0.24 (8B) and 0.28 (32B), compared with -0.03 for bge-small sentence embeddings [Xiao et al., 2024]. In the simulator, the probe scores improved Brier ALC by 0.003 (8B) and 0.005 (32B).

Benchmark difficulty. Without examples, both Qwen3 models rated every training benchmark between 6.8 and 7.7 out of 9 on average, although the benchmarks' mean success rates over all items range from 0.22 to 0.62. With nine anchor items from the other benchmarks and their observed success rates in the prompt, only Qwen3.8-27B, the most recent of the four models, produced a benchmark-level prior better than the constant of HBS (budget-0 Brier score 0.217 against 0.233). Qwen3-8B, Qwen3.5-9B [Qwen Team, 2026a] and the larger but older Qwen3-32B did not (Table 11 in Appendix F), and only Qwen3.8-27B was later measured on the test benchmarks. A single rating is noisy, but a predictor can average its ratings over a benchmark's items. Four training benchmarks are too few to validate this, so we evaluated it on the test benchmarks (Section 6.4).

Item-level shift on real feedback. To test the item-level signal on real data, we submitted HBS+ItemLLM, which shifts the HBS estimate in logit space by $\gamma(z_i - c)$, where z_i is the item's standardized Qwen3-32B probe score, c is the mean score of the target and the labeled items of its benchmark, and $\gamma = 0.5$. It improved Brier ALC by 0.005 in the simulator, with γ selected leave-one-benchmark-out, and ran as intended on the platform, yet it scored 0.157 on its real draw, where the proxy expects 0.131 for HBS, a difference of 3.8 standard deviations of label noise. The loss falls from 0.070 at budget 1 to 0.024 at budget 7 and is near zero from budget 15 (Figure 5b in Appendix F). About 70% of it comes from the two pairs of one benchmark, whose shifts are centered on the same labeled items, which points to a cause on that benchmark, such as extreme scores

among its first revealed items. Scores like those on the training benchmarks would leave HBS nearly unchanged, and the aggregated feedback cannot identify the cause (Appendix F).

6.4 A benchmark prior from an LLM

The measurement submission (Section 5.3) recovered the prior of all seven test benchmarks. The priors order the benchmarks' mean success rates across our reports with Kendall's $\tau = 0.62$ and are closer to them than the constant 0.35 for five of the seven, but they are compressed toward the middle (Appendix E). Used from budget 1, the prior improves on HBS by 0.004. Used from budget 0, where it accumulates item by item, it improves on HBS by 0.007 (95% interval 0.006 to 0.009; Table 7 in Appendix E), mostly at budgets 0 to 3 (Figure 2). This is 44% of what knowing each benchmark's rate gains at the same α, and 39% with $\alpha = 6$. On the 26 pairs with known roots, the improvement is 0.008 and holds in each of their three reports. Removing any one benchmark leaves 0.006 to 0.009, but treated as a sample of benchmarks, the seven give a jackknife standard error of 0.0033, so an approximate 95% interval for new benchmarks reaches from -0.015 to $+0.001$. It is robust to K_0, the rating order and redraws of each prior (Appendix E). These estimates are in-sample with respect to the test pool, because the same seven benchmarks supplied the priors and make up the 69 pairs. On the training benchmarks, whose rates lie closer to 0.35, the same procedure gained only about 0.002 in simulation, which suggests that a benchmark prior helps most when rates are extreme, as in the test pool.

The formative run of the final submission is a check on a fresh draw (Table 9 in Appendix E). At budget 0, which involves no labels and hence no label noise, its Brier score was 0.190, against 0.224 for the HBS constant on the same pairs. Over all budgets it scored 0.174, between the proxy's expectations for HBS (0.180) and HBS+Prior (0.171) and within label noise of both.

7 Discussion and limitations

Benchmark-level cold start is limited by information. How difficult a benchmark is for current systems depends on grading strictness, the harness and environment, and recent progress in model capabilities, none of which is stated in its items. Success rates in the test pool cluster by benchmark, so the first few labels are highly informative, whereas a text-based prior helps only if it reflects current capabilities. In our experiments, LLMs judged a task type's difficulty better than an item's.

Simulation can mislead. Two changes that helped on the public training pool hurt on real feedback, and both relied on statistics learned across that pool. The subject-ability adjustment assumed its older population of systems, and the item-level shift assumed that difficulty signals from four benchmarks transfer. Leave-one-benchmark-out validation on so few benchmarks cannot detect such shifts, so we adopted a change only if it also held on real feedback.

Limitations. The proxy omits item and group structure, treats labels as independent Bernoulli draws, is pessimistic by 0.006 to 0.010 (Table 1) and cannot evaluate text-based methods. Its intervals hold the seven benchmarks and the 43 inferred roots fixed; the comparisons with HBS without pooling and the own-label predictor, and the share due to benchmark difficulty, survive the estimated root errors but not uniform flips at the rule's upper error bound (Appendix E). The 69 pairs come from seven test benchmarks, and the hidden final test subset may differ. The verdict on the item-level shift rests on one real draw, although five of its nine pairs lost. The LLM prior was measured once, on the seven test benchmarks, and without a GPU the final submission falls back to HBS. We tested open models of at most 32 billion parameters, and larger or proprietary models may do better.

8 Conclusion

In benchmark-level cold start, the main limitation is missing information about the new benchmark, not the predictor. No label-only alternative we tested improves on hierarchical shrinkage by more than 0.0004, and more than half of its remaining error comes from not knowing an unseen benchmark's difficulty. On the seven test benchmarks, averaging a recent open LLM's ratings over a benchmark's items supplies about two fifths of this missing prior; its value for new benchmarks is uncertain. A reliable judge of benchmark difficulty would be worth up to 0.019 Brier ALC here.

Acknowledgments and Disclosure of Funding

This study was supported in part by the National Science and Technology Council (NSTC), Taiwan, under grants 115-2627-M-006-001, 115-2218-E-A49-023, and 115-2634-F-A49-014. We thank the National Center for High-performance Computing (NCHC) of the National Applied Research Laboratories (NARLabs) in Taiwan for providing computational and storage resources. The author declares no competing interests.

References

aghasalim. A shrunk logistic predictor for cold start benchmarks. https://github.com/aghasalim/predictive-eval, 2026. Technical report, Predictive AI Evaluation Competition.

AIMS Foundations. measurement-db. https://huggingface.co/datasets/aims-foundations/measurement-db, 2026. Revision bc8204d.

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 2017.

Eliya Habba, Itay Itzhak, Asaf Yehudai, Yotam Perlitz, Elron Bandel, Michal Shmueli-Scheuer, Leshem Choshen, and Gabriel Stanovsky. Growing pains: Extensible and efficient LLM benchmarking via fixed parameter calibration. *arXiv preprint arXiv:2604.12843*, 2026.

Stefan Krsteski and Charlotte Meyer. Predicting task difficulty without rollouts. *arXiv preprint arXiv:2608.05797*, 2026.

Peiyu Li, Xiuxiu Tang, Si Chen, Ying Cheng, Ronald Metoyer, Ting Hua, and Nitesh V. Chawla. Adaptive testing for LLM evaluation: A psychometric alternative to static benchmarks. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. arXiv:2511.04689.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1980.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.

Allan H. Murphy. A new vector partition of the probability score. *Journal of Applied Meteorology*, 12(4):595–600, 1973.

Jungsoo Park, Ethan Mendes, Gabriel Stanovsky, and Alan Ritter. Anticipatory evaluation of language models: Predicting LLM benchmark scores from descriptions. *arXiv preprint arXiv:2509.20645*, 2025.

Qwen Team. Qwen3.5-9B. https://huggingface.co/Qwen/Qwen3.5-9B, 2026a.

Qwen Team. Qwen3.8-27B. https://huggingface.co/Qwen/Qwen3.8-27B, 2026b.

Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *Advances in Neural Information Processing Systems*, 2024.

Stanford AI Measurement Science. The predictive AI evaluation competition. https://aims.stanford.edu/competition, 2026.

Sang T. Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025. URL https://proceedings.mlr.press/v267/truong25c.html.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yuchen Zeng and Dimitris Papailiopoulos. You don't need to run every eval. *arXiv preprint arXiv:2606.24020*, 2026.

Lexin Zhou, Lorenzo Pacchiardi, Fernando Martínez-Plumed, et al. General scales unlock AI evaluation with explanatory and predictive power. *arXiv preprint arXiv:2503.06378*, 2025.

A Submissions

Table 3 lists the formative submissions referred to in this paper. In Figure 3, *Early 1* to *Early 4* denote the four early submissions in the order listed. Their predictors are not evaluated in this paper, but their feedback reports contribute 34 pairs to the proxy.

Table 3: Formative submissions and their Codabench identifiers.

Method	Description	Codabench ID
Early submissions	Our first predictors, made before the methods in this paper; code not retained	928061, 930136, 930165, 931142
Smoothed mean	Pair's label mean shrunk ($\alpha = 5$) toward the benchmark's label mean, itself shrunk toward 0.35 with strength 30; with the item correction	951748
HBS+Ability	HBS with a prior mean of 0.4 and a subject-ability shift estimated on the training data	951853
HBS	Section 4; label-only	952533
HBS+ItemLLM	HBS with an item-level shift from Qwen3-32B representations (Section 6.3)	960385
Measurement	Measures the LLM benchmark prior on the test benchmarks (Section 5.3); not a candidate for the final ranking	978024
HBS+Prior	HBS with the LLM benchmark prior from budget 0 (Section 4); final submission	978186

Smoothed mean. With k_b and n_b the successes and labels of all subjects on benchmark b, including the target subject, the pair estimate is $(k_{sb}{+}5m_b)/(n_{sb}{+}5)$ with $m_b = (k_b{+}30{\cdot}0.35)/(n_b{+}30)$. It adds the item correction of Equation (4) with $\kappa_I = \gamma_I = 1$, where residuals are taken against the same pair estimates, and clips to $[0.01, 0.99]$; it has no group correction.

HBS+Ability. It uses Equation (3) with $\mu_0 = 0.4$ and moves π_{sb}, clipped to $[0.01, 0.99]$, by βa_s in logit space before the pair update, with $\beta = 0.55$. The ability a_s comes from the training pairs with at least 80 items, fitted by alternating least squares in $\mathrm{logit}(r_{sb}) = d_b + a_s$ with rates clipped to $[0.01, 0.99]$ and abilities centered. A test subject whose model name, or the source model name in its extra features, was seen in training receives that model's ability; otherwise, a_s is predicted by a ridge regression (strength 1, unpenalized intercept) on the centered release year and the reasoning effort on an ordinal scale from 0 to 5, each with an indicator for a missing value.

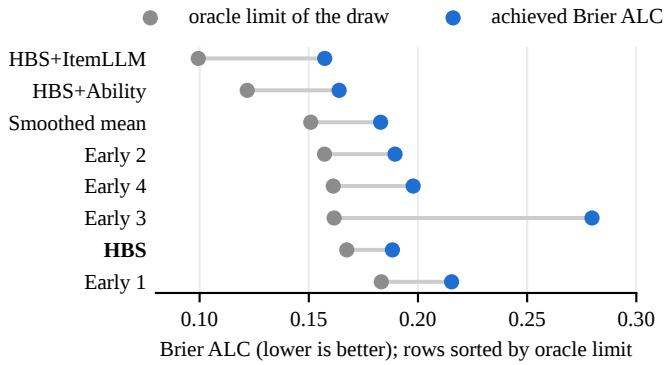


Figure 3: Oracle limit and achieved Brier ALC of the eight formative submissions used to build the proxy. The oracle limit measures the difficulty of each draw, independently of the method.

B Prompts

All prompts were formatted with the model's chat template, with thinking disabled, followed by the prefilled answer prefix "`Rating:␣`", so that the next token is the rating. We read the probabilities of the digit tokens 0 to 9 at that position and renormalize them. The rating is the expected digit $\mathbb{E}[d]$; for the anchored prompt, it is mapped to a success rate $(\mathbb{E}[d] + 0.5)/10$. The item-difficulty probe uses the hidden states of the zero-shot prompt rather than its rating. `{content}` denotes the item text. A text longer than its limit L keeps its first h tokens and a window of $L - h$ further tokens, joined by "[...]" (h = 384 of 2,048 tokens for zero-shot ratings, 64 of 256 for anchors and 192 of 1,024 for targets). The window is the end of the text, unless a `TODO` marker starts before it (the first occurrence of `TODO: Implement block`, or else of `TODO`, which marks the code to write in code-completion items). Then, with the marker at token m and $a = \min(512, \lfloor (L - h)/3 \rfloor)$, the window starts at $\max(h, m + a - (L - h))$, so it ends a tokens after the marker unless that would overlap the head. Token positions are counted with each model's own tokenizer on the item text before it is inserted into the prompt.

Zero-shot rating (item difficulty). Item texts were truncated to 2,048 tokens.

> Below is a task from an AI evaluation benchmark. A capable AI system (a frontier language model, possibly acting as an agent with tools) will attempt it, and the attempt is graded pass/fail.
>
> `<task>`
> `{content}`
> `</task>`
>
> Do not attempt the task yourself. How likely is a strong AI system to solve this task correctly? Answer with a single digit from 0 to 9, where 0 means it almost certainly fails and 9 means it almost certainly succeeds.

Anchored rating (benchmark difficulty). The introduction is followed by the examples in a fixed random order and then by the question: nine examples from the other three benchmarks in the experiments of Section 6.3, and twelve from all four benchmarks in HBS+Prior, a prompt that therefore cannot be evaluated leave-one-benchmark-out on the training benchmarks. The anchors of a benchmark are the items at positions $\lfloor q(N - 1) \rfloor$, $q \in \{0.2, 0.5, 0.8\}$, among its N items with at least five subjects sorted by success rate and then by item identifier, so the 20th, 50th and 80th percentiles; the examples appear in an order shuffled once with seed 0. `{pct}` is the success rate as a rounded percentage, and `{digit}` is $\min(9, \lfloor 10 \times \text{rate} \rfloor)$ of the unrounded rate. In Section 6.3, the targets are 120 random items per benchmark with at least three subjects (seed 0); HBS+Prior rates every item it is asked about.

> You will estimate how often AI systems succeed on a task from an AI evaluation benchmark. The AI systems are language models, often acting as agents with tools, and each attempt is graded pass/fail.
>
> For reference, here are tasks from other benchmarks with the share of AI systems that solved them. A digit d means d*10% to (d+1)*10%.
>
> `<example>`
> `<task>`
> `{content}`
> `</task>`
> Share of AI systems that solved it: `{pct}`% (digit `{digit}`)
> `</example>`
>
> [... eight further examples (eleven in HBS+Prior) ...]
>
> Now a new task, from a different benchmark:
>
> `<task>`
> `{content}`
> `</task>`
>
> Do not attempt the task yourself. What share of AI systems solve this task? Answer with a single digit from 0 to 9 as defined above.

C Reproducibility and implementation details

Code and data. Code is available at https://github.com/kay0601/how-far-can-labels-go; its README lists the command that produces each table and figure. The training data are revision bc8204d of measurement-db, obtained with the official preparation script; `matharena`, which that script excludes, is downloaded separately at the same revision. Formative feedback is private to each team and is not released; the proxy scripts read a team's own scoring logs. The official preparation script excludes `matharena` because about 60 of its responses are not binary; we drop those responses and keep the rest. `mmdocrag` is not used because its scores are continuous, and `swe_rebench`, with one subject per item, is used for training only.

Feature groups. `item_features` is a list of `key=value` fields separated by semicolons. The group of an item is its benchmark together with the sorted list of its fields whose value is not a number, except `prompt_source`; a value counts as a number if it consists of digits with at most one decimal point. An item without such fields has no group and no group correction. Labels on item i itself from other subjects enter both $R(i)$ and $G(i)$. For a single subject on a benchmark whose items share one group, Equation (4) turns $\bar{p}_{sb}$ into the shrinkage estimate $(k_{sb} + \alpha'\pi_{sb})/(n_{sb} + \alpha')$ with $\alpha' = \alpha\,(n_{sb}(1-\gamma_G) + \kappa_G)/(n_{sb} + \kappa_G + \alpha\gamma_G)$, which is 1.9, 1.7, 1.4, 1.2 and 1.0 at 1, 3, 7, 15 and 31 labels instead of $\alpha = 3$.

Simulator. Each simulated draw adds pairs until it holds 1,000 items: a random training benchmark and subject, with 88 to 230 of its items chosen uniformly at random. With probability 0.15, a second subject of the same benchmark joins the pair's items if it shares at least 80 of them. Stratified acquisition replays the platform's stream of candidate items: a candidate is revealed if its feature group has no more labels from the pair than the least sampled group so far, or if every remaining candidate is needed to fill the budget; items without a group are always revealed.

HBS+Prior. The platform calls the predictor sequentially in one worker process and restarts it at each budget, so ratings are written to disk as they are made and reloaded after each restart; each item is rated only once. Only items to be predicted are rated, never revealed acquisition items, and the target item is rated before its prior is computed, so the k-th distinct item of a benchmark at budget 0 sees $|I_b| = k$. Each benchmark's sum and count of ratings are shared by all subjects on it, so at budget 0 the prior used for an item depends on how many items of its benchmark, including other subjects' items, were rated before it. The platform calls the predictor once per evaluation item and does not reveal the remaining items, so the prior cannot be computed over all items before the first prediction. Rating one item takes about 3 seconds on an RTX 3090, well within the platform's limit of 120 seconds per call; the 4-bit model needs about 18 GB of GPU memory and is downloaded by the platform from the model list of the submission. If the model cannot be loaded, the submission falls back to HBS.

LLM experiments. The ridge probe of Section 6.3 uses the hidden state of the last prompt token at half, three quarters or all of the model's depth, or the mean over tokens at half depth, with a regularization strength of 10^3 or 10^4. The best setting is 10^4 with layer 27 of 36 for Qwen3-8B and with the final layer, 64, for Qwen3-32B. Qwen3-32B and Qwen3.8-27B were loaded in 4-bit precision (bitsandbytes NF4 with double quantization and bfloat16 computation), and Qwen3-8B and Qwen3.5-9B in bfloat16. The 5,322 rated items are all items of the four multi-subject training benchmarks and 1,000 random items of `swe_rebench` (seed 0). For HBS+ItemLLM, z_i is the probe's prediction divided by the standard deviation of its leave-one-benchmark-out predictions on the training items.

Compute. One simulator configuration (400 draws) takes about 10 seconds on a CPU. The LLM experiments ran on a single RTX 3090 (24 GB): extracting Qwen3-8B representations for 5,322 items took 12 minutes and Qwen3-32B representations about 50 minutes; the anchored ratings took 10 minutes for Qwen3.5-9B, 22 minutes for Qwen3-32B and 28 minutes for Qwen3.8-27B. The formative run of the measurement submission, with 491 evaluation items, completed in under 40 minutes. Software: Python 3.13 for HBS and the simulator; Python 3.10 with PyTorch 2.7.1, Transformers 5.2.0 and bitsandbytes 0.50.2 for the LLM experiments.

Table 4: Public training benchmarks (measurement-db, revision bc8204d). Only benchmarks with several subjects per item can be used to evaluate methods offline.

Benchmark	Items	Subjects per item (median)	Use
`multi_swebench`	2,126	27	training and offline evaluation
`matharena`	1,751	9	training and offline evaluation
`real_webagents`	233	12	training and offline evaluation
`researchcodebench`	212	31	training and offline evaluation
`swe_rebench`	6,306	1	training only

Table 5: Hyperparameters of HBS, with their names in the released code.

Symbol	Code name	Role	Value
μ_0	`PRIOR_MEAN`	mean of the global prior	0.35
κ_0	`GLOBAL_K`	strength of the global prior	100
κ_1	`BENCH_K`	strength of the benchmark prior	3
α	`ALPHA`	strength of the pair prior	3
κ_I, γ_I	`ITEM_K`, `ITEM_GAMMA`	shrinkage and scale of the item correction	1, 1
κ_G, γ_G	`GROUP_K`, `GROUP_GAMMA`	shrinkage and scale of the group correction	5, 0.75
ϵ	`CLIP`	clipping margin	0.01

D Disclosures

- Training data: only the public training pool of measurement-db (CC BY-SA 4.0, gated access under its terms of use), as listed in Table 4. We do not redistribute the data. No external data and no contributions from other teams were used.
- Pretrained models: BAAI/bge-small-en-v1.5 (MIT license) and Qwen/Qwen3-8B, Qwen/Qwen3-32B, Qwen/Qwen3.5-9B and Qwen/Qwen3.8-27B (Apache 2.0). The final submission (HBS+Prior) loads Qwen3.8-27B at run time; HBS uses no pretrained model; HBS+ItemLLM loaded Qwen3-32B.
- Coding assistant: we used Claude Code (Anthropic) for implementation, analysis and drafting this report.
- Broader impact: predictive evaluation can reduce the compute spent on evaluating AI systems. Its main risk is that predicted success rates are trusted in place of measured ones; our results show that predictions are least reliable on unseen benchmarks before labels arrive, which is where such trust would be most tempting. Scores on small random samples also mix the quality of a method with the difficulty of the sample, which matters for anyone who ranks methods on such samples; reporting each draw's oracle limit next to its score, which any team can compute from its own feedback, would separate the two.

E Robustness of the proxy

Subsets. The 26 pairs with known roots come from the reports of the smoothed mean, HBS and HBS+ItemLLM, which predict 0.35 at budget 0. The 26 pairs of Table 1 and Figure 4, a different set, come from the reports of the three submissions whose code we retained, the smoothed mean, HBS+Ability and HBS; for HBS+Ability, the proxy uses the inferred budget-0 predictions, through an ability table that reproduces them. HBS+ItemLLM predicts 0.35 at budget 0 because its shift vanishes without labels ($c = z_i$). For HBS+Ability, both the root and the sign of $p_j - r_j$ are inferred: for seven of its nine pairs only one sign gives a prediction in $[0, 1]$, and for the other two we take the higher candidate, which differs from the lower one by 0.08 and 0.03. For all three submissions of Table 1, the budget-0 scores match the proxy by construction, so the table tests budgets 1 to 31. The reference of an inferred pair is the mean rate of its benchmark over the 26 known pairs; the roots of HBS+Ability are inferred first, and for the four early reports the reference also includes its pairs. Every test benchmark appears in all eight reports and contributes 8 to 11 of the 69 pairs and 3 to 5 of the 26 known; 26 of the 69 pairs, 10 of the 26 known, share their benchmark with another subject in the same draw.

Pair-level validity. Figure 4 compares each pair's real Brier ALC with the proxy's expectation for the three submissions of Table 1, as a z-score under label noise alone. Of the 25 pairs with label noise, 24 lie within two standard deviations, and the mean z of -0.35 reflects the proxy's pessimism. The exception ($z = 3.6$, on the draw of HBS) is a pair whose real Brier score rose from 0.27 at budget 0 to 0.34 at budgets 3 to 15. This is consistent with an acquisition half whose success rate differs from that of the evaluation half, which the proxy does not model because it draws labels from the evaluation rate. The one pair without label noise, whose rate is 0 or 1 so that every simulated label sequence is the same, matches its expectation to within 10^{-6}.

Pessimism of the proxy. The real scores of Table 1 fall 0.006 to 0.010 below the proxy. The item and group corrections, which the proxy disables, are together worth 0.0015 in the simulator, the group correction alone 0.0011 (Table 2). The finite pools of items do not explain the rest. Drawing each pair's labels without replacement from a pool of $|A_j| = |E_j|$ items at exactly the evaluation rate r_j lowers the proxy's expectation by 0.0008 to 0.0009, but under a random split of a pair's items the acquisition half's rate itself varies around r_j. Drawing it given r_j, with a uniform prior on the pair's underlying rate, instead raises the expectation by 0.0022 to 0.0030, and the real scores then lie 1.0 to 1.5 standard deviations below it. In either model, the remaining difference is consistent with favorable label sequences on the three draws. A difference between the rates of the two halves raises a real score above the proxy, as for the outlying pair above.

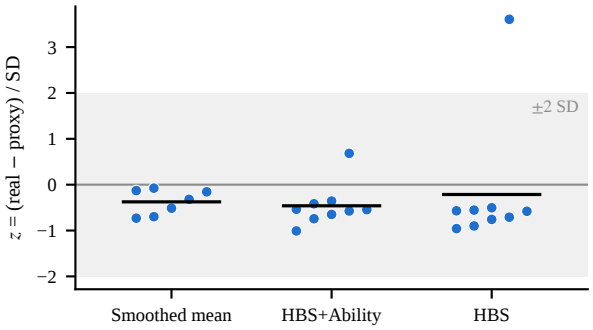


Figure 4: Real minus proxy-expected Brier ALC of each pair with label noise, in standard deviations of label noise, for the three submissions of Table 1. Bars mark the mean of each submission; the shaded band is ± 2 standard deviations.

Reports that did not inform the design. Because the reports of the smoothed mean and HBS+Ability informed the design of HBS, we repeated the comparisons of Table 2 with the official baseline, the smoothed mean, the ablation and the own-label predictor on the other six reports: every difference keeps its sign and changes by at most 0.0012.

Choice of roots. For the 43 pairs whose budget-0 prediction is unknown, from the four early reports and the report of HBS+Ability, we chose between r_j and $1 - r_j$ using the rates of the same benchmark in the reports whose prediction is known. Applied to the 26 pairs whose root is known, with references from the other known reports, this rule picks the correct root for 25 and lands within 0.1 of it for all 26; every benchmark appears in all three known reports, and the miss is a pair near 0.5, where the two roots nearly coincide. Because the rule picks the root closer to the benchmark's other rates, it can only tighten the clustering of rates by benchmark, which may explain part of the higher share on all 69 pairs than on the 26 known. Table 6 repeats the main comparisons of Table 2 on the 26 known pairs alone, with all 43 inferred roots flipped, and under two models of random errors, each with 40 assignments of 300 label sequences. Uniform flips flip each inferred root with probability 0.20, the upper end of the 95% interval for the rule's error rate. Weighted flips flip each root with its probability of being wrong if rates scatter normally around the mean rate of their benchmark over the known pairs, with the standard deviation of the known pairs around these means, 0.115. This probability exceeds 0.1 for nine of the 43 pairs, all with rates within 0.1 of 0.5, where a flip changes the rate by at most 0.2, and the expected number of flips is 2.1. The advantages of HBS over the official baseline and the smoothed mean hold in every case, and under weighted flips

every comparison keeps its sign. The advantage over HBS without benchmark pooling holds on the known pairs and with all roots flipped, but reverses in some uniform assignments. The comparison with the own-label predictor and the share of the gap due to benchmark difficulty hold on the known pairs, where the share is 56%, but reverse when all 43 roots are flipped, which would contradict the rates of the same benchmarks in other reports, and become uncertain under uniform flips, which break the clustering of rates by benchmark that the rule relies on. The intervals of Tables 2 and 7 hold the inferred roots fixed.

Benchmark-difficulty diagnostic. The diagnostic replaces μ_0 by the benchmark's mean rate over the pairs of the other seven reports, in each of which every test benchmark appears; κ_0 and κ_1 keep their values, so labels on other benchmarks and of other subjects on the same benchmark still dilute this mean. It is estimated from 7 to 10 pairs, so part of the remaining gap is its estimation error. With $\alpha = 3$, it scores 0.162 and closes 58% of the gap between HBS and the oracle (95% interval 54 to 61%); fixing g_b at that mean instead ($\kappa_0 \to \infty$) closes 59%. With $\alpha = 6$, used in Figures 1 and 2, it closes 65%. On the 26 known pairs, the shares are 54% and 56%.

Table 6: Change in Brier ALC of each alternative relative to HBS, and the share of HBS's gap to the oracle closed by knowing each benchmark's rate ($\alpha = 6$), under different choices of the 43 inferred roots. Uniform flips flip each root with probability 0.20; weighted flips use each root's estimated probability of error.

Roots	Baseline	Smoothed mean	No pooling	Own-label	Share
As inferred, 69 pairs (Table 2)	+0.028	+0.003	+0.003	+0.002	0.65
26 known pairs only	+0.024	+0.005	+0.005	+0.003	0.56
All 43 flipped	+0.017	+0.010	+0.006	−0.004	−0.12
Uniform flips, minimum over 40	+0.019	+0.001	−0.001	−0.004	−0.12
Uniform flips, maximum over 40	+0.029	+0.004	+0.004	+0.003	0.53
Weighted flips, minimum over 40	+0.027	+0.003	+0.003	+0.002	0.62
Weighted flips, maximum over 40	+0.028	+0.003	+0.003	+0.002	0.65

HBS+Prior. Table 7 gives the effect of the LLM prior on the 69 pairs. From budget 1 onward, the proxy uses the measured priors. At budget 0, it models the prior for the k-th evaluation item of a pair as the K_0-shrunk mean of k ratings drawn around the benchmark's mean rating with the standard deviation of single ratings on the training benchmarks, 0.205; the mean rating is recovered by undoing the shrinkage of μ_b over the n_b evaluation items of benchmark b in the measurement run, summed over its pairs, and $|E_j|$ is read from each report. Each pair is simulated as if its items were rated first, which slightly understates the prior's accumulation when two pairs share a benchmark, the simulated ratings are drawn independently rather than constrained to the recovered mean, and the model ignores any correlation between an item's rating and its outcome. From budget 1, every report uses the measured μ_b. Two benchmarks had two pairs in the measurement run; because an item shared by both pairs is rated once, n_b may count it twice, which would change the recovered mean rating by at most 0.010. The variants with the prior from budget 0 and from budget 1 differ only at budget 0, whose weight is 0.1: its gain of 0.030 accounts for their difference of 0.003. Placing μ_b in the global rate follows the structure of HBS; fixing g_b at μ_b instead ($\kappa_0 \to \infty$), so that labels on other benchmarks do not dilute it, improves the proxy score by a further 0.0006. In the measurement run, r_j is known from budget 0, and the budget-1 error $e = |\mu_b - r_j|$ leaves the candidates $\mu_b = r_j \pm e$, whose budget-3 predictions have ECE values $|0.1 - e|$ and $0.1 + e$; these differ whenever $e > 0$, and no prior exceeded 0.89, so $\mu_b + 0.1$ was never clipped. Equivalently, $\mathrm{Brier}_j(3) - \mathrm{Brier}_j(1) = 0.01 + 0.2(\mu_b - r_j)$ gives the sign directly.

Accuracy of the priors. Across the seven test benchmarks, the priors order the benchmarks' mean success rates with Kendall's $\tau = 0.62$ and lower the mean absolute error from 0.22 for the constant 0.35 to 0.15, but they are compressed: regressing the priors on the mean rates gives a slope of 0.38. The prior lowers the expected Brier ALC on five of the seven benchmarks and on 51 of the 69 pairs; removing any one benchmark leaves an improvement of 0.0057 to 0.0088, and the jackknife standard error over benchmarks, 0.0033, is larger than the interval of Table 7 suggests. Table 8 collects the checks of Section 6.4. The priors were redrawn 20 times from their sampling distribution (the mean of n ratings with standard deviation 0.205, where n is the number of items rated in the measurement

run); the order of the items was permuted 200 times for fixed ratings; and K_0, set to 5 before the measurement, was varied from 0 to 20. In the formative run of the final submission, predictions at budgets 1 and 3 are constant within five of the nine pairs and nearly constant within the other four, where only the corrections of Equation (4) vary, so both budgets approximately identify each pair's rate: Equation (6) applied to budgets 1 and 3 gives estimates of $r_j(1 - r_j)$ that agree exactly for the five pairs and to within 0.004 for the others. Of the two roots, we take the one closer to the benchmark's mean rate over the 69 pairs. The close agreement of the two budgets also indicates that the small variation within the four pairs hardly affects Equation (6). Table 9 compares that run with the proxy's expectations on its pairs.

Table 7: HBS with the LLM benchmark prior on the 69 real pairs. The priors are fixed at the values measured in Section 5.3, so the 95% intervals, from resampling whole reports, cover label noise and the sampling of pairs, but neither the sampling error of the priors (Table 8) nor benchmarks outside the test pool; for the final submission, a jackknife over the seven benchmarks gives a standard error of 0.0033 and an approximate 95% interval from -0.015 to $+0.001$. Changes are computed before rounding; negative changes are improvements.

Predictor	Brier ALC	Change from HBS
HBS	0.179	—
HBS+Prior, prior used from budget 1	0.175	-0.0044 $[-0.0052, -0.0035]$
HBS+Prior, prior used from budget 0 (final submission)	0.171	-0.0073 $[-0.0087, -0.0059]$
Each benchmark's rate known, $\alpha = 3$ (diagnostic)	0.162	-0.0166 $[-0.0185, -0.0149]$
Each benchmark's rate known, $\alpha = 6$ (diagnostic)	0.160	-0.0186 $[-0.0204, -0.0171]$

Table 8: Change in Brier ALC of HBS+Prior (prior from budget 0) relative to HBS on the 69 pairs under different checks. Negative changes are improvements.

Check	Change in Brier ALC
As submitted ($K_0 = 5$), 95% interval over reports	-0.0073 $[-0.0087, -0.0059]$
26 pairs with known roots	-0.0082
$K_0 = 0$, 2, 10, 20	-0.0078, -0.0077, -0.0069, -0.0061
Item order permuted 200 times	-0.0073 (SD 0.0001)
Priors redrawn 20 times	-0.0073 (SD 0.0008)

Table 9: The formative run of the final submission against the proxy's expectations for HBS and HBS+Prior on its nine pairs. The budget-0 expectation for HBS involves no labels and hence no label noise; the label-noise standard deviation of the Brier ALC is 0.007.

Budget	0	1	3	7	15	31	Brier ALC
Observed, final submission	0.190	0.192	0.183	0.163	0.163	0.151	0.174
Expected, HBS	0.224	0.201	0.181	0.167	0.160	0.156	0.180
Expected, HBS+Prior	0.194	0.182	0.173	0.165	0.159	0.155	0.171

Baselines and tuning grids. The official baseline predicts the mean of the pair's revealed labels, and 0.5 without labels. The own-label Bayes predictor takes as its prior the success rates $r_{j'}$ of the other 68 pairs, with equal weights, and predicts the posterior mean $\sum_{j'} r_{j'}^{k+1}(1 - r_{j'})^{t-k} / \sum_{j'} r_{j'}^{k}(1 - r_{j'})^{t-k}$ after k successes in its own t labels. HBS without pooling is 0.0006 worse than this predictor, so HBS beats it through pooling. The mixture prior replaces the pair prior of HBS, $\mathrm{Beta}(\alpha\pi_{sb}, \alpha(1 - \pi_{sb}))$, by a mixture with a near-unsolvable component $\mathrm{Beta}(km, k(1 - m))$ of weight w, and predicts the exact posterior mean, reweighting the components by their Beta-Binomial likelihoods of the pair's labels; we search $w \in \{0, 0.1, \ldots, 0.4\}$, $m \in \{0.02, 0.05, 0.1\}$ and $k \in \{5, 10, 30\}$. Re-tuning searches $\mu_0 \in \{0.3, 0.35, 0.4\}$, $\alpha \in \{2, 3, 4, 6\}$ and $\kappa_1 \in \{1, 3, 6\}$. Both select leave-one-report-out.

F LLM experiments

Tables 11 and 12 and Figure 5a give the anchored benchmark-level ratings of Section 6.3 on the four held-out training benchmarks, and Figure 5b compares HBS+ItemLLM with the proxy at each budget. Most of the gain of Qwen3.8-27B on these benchmarks comes from `multi_swebench`, and its estimates are compressed toward the middle, as on the test benchmarks (Appendix E). Four benchmarks are too few to select a model reliably; only Qwen3.8-27B was measured on the test benchmarks, and the twelve-anchor prompt of HBS+Prior could not be validated on the training benchmarks.

HBS+ItemLLM. With L the labeled items of the target's benchmark, from any subject, $c = (z_i + \sum_{k \in L} z_k)/(|L|+1)$, so the shift equals $\gamma\,|L|\,(z_i - \bar{z}_L)/(|L|+1)$ with $\gamma = 0.5$ at every budget, and it vanishes at budget 0. On the four multi-subject training benchmarks, the leave-one-benchmark-out probe scores, standardized as in HBS+ItemLLM (Appendix C), have a within-benchmark standard deviation of 0.59 to 1.02 and a mean within-subject correlation with the outcomes of 0.12 to 0.23. On the real draw, six of the nine pairs score worse than the proxy expects for HBS at each of budgets 1, 3 and 7, and the two subjects of one benchmark account for about 70% of the loss; their shifts are centered on the same labeled items, the labels of both subjects on that benchmark. At budget 31, three pairs lose and one gains by more than five standard deviations of label noise.

Table 10 simulates the shift on the same nine pairs, with their recovered rates and evaluation sizes, against HBS on the same labels (300 label sequences). Each item receives an outcome and a synthetic score $z = \sigma(\rho u + \sqrt{1-\rho^2}\,e)$, where u is the outcome standardized within the pair and e is standard normal; unless stated otherwise, the scores are independent across items. Scores like those on the training benchmarks leave HBS nearly unchanged. Uninformative scores, even for 5% of the items or on a quarter of the benchmarks, cause losses that persist to budget 31. Independent scores reproduce the average pattern only if they spread more than ten times as widely as on the training benchmarks and correlate with the outcomes at about 0.8, four times the training value; such scores push predictions toward 0 or 1, and an inaccurate c moves whole pairs the wrong way until enough labels arrive. We consider this regime implausible for a probe that ranks training items with a Spearman correlation below 0.3, and an artifact of assuming the same independent scores for every item. A cause local to the benchmark that carries most of the loss fits better: with training-like scores elsewhere, a single score of 12 at its first revealed item raises the mean Brier score by 0.063 at budget 1. This effect fades faster than observed, to 0.014 at budget 3 and 0.001 at budget 7, because that item's weight in c falls as $1/(|L|+1)$; several extreme scores among the early items would fade more slowly. The aggregated feedback cannot distinguish these explanations, and none of them requires the probe to have transferred.

Table 10: Change in Brier score of HBS+ItemLLM relative to HBS at budgets 1 to 31 on its draw: the real run against the proxy's expectation for HBS, and synthetic scores with spread σ and correlation ρ with the outcomes. On the training benchmarks, σ is 0.59 to 1.02 and ρ is 0.12 to 0.23.

Item scores	1	3	7	15	31
Real run	+0.070	+0.040	+0.024	−0.000	−0.002
$\sigma = 1$, $\rho = 0.2$	−0.002	−0.003	−0.003	−0.004	−0.004
$\sigma = 1$, $\rho = 0.2$; score 12 at the first revealed item of the benchmark with most of the loss	+0.063	+0.014	+0.001	−0.003	−0.003
$\sigma = 3$, $\rho = 0$	+0.050	+0.054	+0.047	+0.041	+0.038
$\sigma = 1$, $\rho = 0.2$; 5% of items with $\sigma = 20$, $\rho = 0$	+0.021	+0.027	+0.021	+0.015	+0.013
$\sigma = 6$, $\rho = 0.6$; each benchmark $\rho = 0$ with probability 0.25	+0.050	+0.047	+0.035	+0.025	+0.020
$\sigma = 4$, $\rho = 0.45$	+0.016	+0.011	+0.002	−0.005	−0.008
$\sigma = 8$, $\rho = 0.7$	+0.033	+0.020	+0.006	−0.007	−0.011
$\sigma = 12$, $\rho = 0.8$	+0.050	+0.031	+0.012	−0.002	−0.008
$\sigma = 16$, $\rho = 0.85$	+0.065	+0.041	+0.018	+0.001	−0.004

Table 11: Budget-0 priors from anchored LLM ratings on the four held-out training benchmarks. Brier is the mean over the four benchmarks of the squared error over all responses to the rated items. Kendall's τ compares the predicted and actual orderings of the benchmarks and is undefined for the single-item rows, which predict $0.35 + w(\rho_i - 0.35)$ for each item with $w \in \{0, 0.25, 0.5, 0.75, 1\}$ chosen in hindsight ($w = 0.5$) or leave-one-benchmark-out.

Budget-0 prior	Kendall τ	Brier
Constant 0.35 (HBS)	—	0.233
Qwen3-8B, mean rating of the benchmark	0.33	0.237
Qwen3.5-9B, mean rating of the benchmark	−0.67	0.256
Qwen3-32B, mean rating of the benchmark	0.00	0.285
Qwen3.8-27B, mean rating of the benchmark	0.67	0.217
Qwen3.8-27B, single item, shrinkage chosen in hindsight	—	0.220
Qwen3.8-27B, single item, shrinkage chosen LOBO	—	0.231
Oracle that knows each benchmark's rate	1.00	0.199

Table 12: Mean predicted success rate of each held-out training benchmark from anchored ratings, against its actual mean rate on the rated items.

	`multi_swebench`	`researchcodebench`	`real_webagents`	`matharena`
Actual	0.125	0.341	0.366	0.645
Qwen3-8B	0.206	0.108	0.378	0.340
Qwen3.5-9B	0.535	0.532	0.459	0.514
Qwen3-32B	0.330	0.384	0.700	0.210
Qwen3.8-27B	0.202	0.359	0.407	0.394

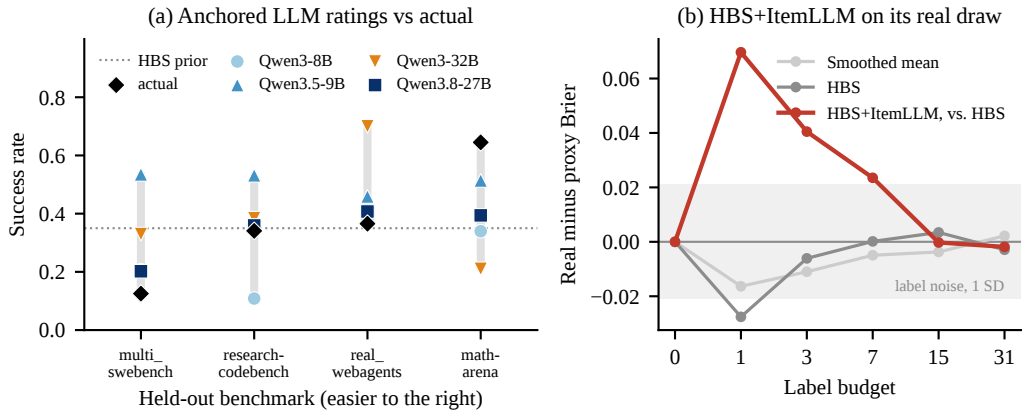


Figure 5: (a) Mean success rate predicted by anchored prompts for each held-out training benchmark (markers) against the actual rate (diamonds); dotted line: the budget-0 prediction of HBS. (b) Real minus proxy-expected Brier score per budget on each submission's own draw; HBS+ItemLLM is compared with the expectation for HBS.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and the introduction (Section 1) state three contributions, supported in Sections 5 and 6 (Proposition 1, Tables 1, 2 and 7, Figures 3 and 2); their scope (one competition, 69 pairs, four training benchmarks) is stated in Section 7.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 (Limitations) covers the proxy's assumptions and pessimism, the inferred roots, the number of pairs and benchmarks, the single real draw for the item-level LLM shift, the single measurement of the LLM prior, and the model sizes we tested.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Proposition 1 states its assumption (a constant budget-0 prediction within the pair) and is followed by a complete proof (Section 5).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4 and Table 5 fully specify HBS and HBS+Prior; Sections 3, 5 and 6.3 and Appendices B and C give the simulator, the proxy, the measurement submission, prompts and data; the repository README gives the exact commands. Reproducing the proxy and the measurement analysis requires a team's own formative logs, which the competition keeps private.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code and instructions are released at `https://github.com/kay0601/how-far-can-labels-go` (Appendix C), and the training data are public (measurement-db, revision bc8204d). Our formative logs are private under the competition rules and are not released; the proxy scripts read a team's own logs.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 gives all hyperparameters and how they were set; Sections 3 and 5 give the simulator and proxy protocols and the held-out selection; Section 6.3 and Appendix B give the LLM settings.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: Table 1 reports label-noise standard deviations and z-scores; Tables 2 and 7 report 95% intervals from resampling whole reports, and Table 2 a simulator standard error; Appendix E reports further robustness checks, including the sensitivity to the inferred roots and a jackknife over the seven test benchmarks.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix C lists the hardware (a CPU for HBS and the simulator; a single RTX 3090 for the LLM experiments and HBS+Prior), the run time of each experiment and the software versions.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses public benchmark data and our own formative feedback as permitted by the competition rules, involves no human subjects, and releases no recovered test data (Section 5.3).

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Appendix D discusses the positive impact (cheaper evaluation) and the negative impact (predicted scores trusted in place of measured ones).

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: We release no data or models with a high risk of misuse; the released code predicts success rates from labels.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: measurement-db (CC BY-SA 4.0, gated), the Qwen models (Apache 2.0) and bge-small-en-v1.5 (MIT) are cited, and their licenses are stated in Appendix D; we do not redistribute the data.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The released code is documented by a README that maps every table and figure to the command that produces it (Appendix C), and is released under the MIT License.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work involves no crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work involves no human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: An LLM (Qwen3.8-27B) is a component of the final submission, HBS+Prior: Section 4 and Appendix B describe the model, prompt and settings, and Sections 6.3 and 6.4 evaluate it and the other LLMs we tested. The use of a coding assistant is disclosed in Appendix D.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.